27/09/25

Data structure in Python :-

Python data structure are way of Organizing and Sorting data so that they can be accessed and modified efficiently.

→ Python provides both built in data structure and allow us to implement user defined data structure.

Built in data structure in Python are:

→ Built in data

list : [ ]
tuple : ( )
set : { }
dict : {key : values}

Note:- if you want to store multiple values in a single variable means:

```
names = input(" enter your names:").split()  - run this
```

for text

```
enter your names : pradeep , ramesh . subbu

names - call
['pradeep', 'ramesh', 'subbu']

num = list (map (int , input ("enter the values :").split()))
enter the values : 18 65 87
```

for int type

```
num - call

[18, 65, 87]
```

1) List ( ) (or) [ ] : the list is a hetrogeneous data collection and its a Ordered, mutable and allow duplicates.

Hetrogeneous data collection : allow to store d/f kinds of data like String, int , float

Ordered :- in which Ordered we store in that Order only it comes.

* any multi valued variable contained a position.

Mutable :- We can make changes to the list i.e to add values after execution or delete values

---

list() (or) [] :-

Example :- n1 = list([2,6,4,5,8])
n1
[2, 6, 4, 5, 8] - o/p
n2 = [5, 8, 0, 8, 2]
n2
[5, 8, 0, 8, 2] - o/p

so for heterogeneous

l1 = ["pradeep", "26-01-2003", 5.9, 1586, True, 5j+2]
l1
['pradeep', '26-01-2003', 5.9, 1586, True, (2+5j)] - o/p

d/f kinds of data

l1[0]
'pradeep'

```
for i in l1:
    print(i)
```

pradeep
26-01-2003
5.9
1586
True
(2+5j)

l1.append(24)
l1
['pradeep', '26-01-2003', 5.9, 1586, True, (2+5j), 24]

```
l1.append([1,2,3])
l1
['pradeep', '26-01-2003', 5.9, 1586, True, (2+5j), 24, [1,2,3]]
l1.pop()  delete One last value
l1
['pradeep', '26-01-2003', 5.9, 1586, True, (2+5j), 24]
l2 = [34.5, 5.9, "pradeep"]
l2
[34.5, 5.9, 'pradeep']
l1 + l2
['pradeep', '26-01-2003', 5.9, 1586, True, (2+5j), 24, 34.5, 5.9, 'pradeep']
l1 = l1 + l2
l1
['pradeep', '26-01-2003', 5.9, 1586, True, (2+5j), 24, 34.5, 5.9, 'pradeep']
l1[1] = "9902611482"
l1
['pradeep', '9902611482', 5.9, 1586, True, (2+5j), 24, 34.5, 5.9, 'pradeep']
l1.insert(2, "08-10-1999")
l1
['pradeep',
 '9902611482',
 '08-10-1999',
 5.9,
 1586,
 True,
 (2+5j),
 24,
 34.5,
```

5.9,

'pradeep']

l1. pop (8)

l1

['pradeep',

'9902611482',

'08-10-1999',

1586,

True,

(2+5j),

24,

84.5,

5.9,

'pradeep']

l1. remove (24)

l1

['pradeep',

'9902611482',

'08-10-1999',

1586,

True,

(2+5j),

84.5,

5.9

'pradeep']

l1. clear()

[ ] - o/p

l1

[ ] - o/p

del l2

l2 - 0

we get error because we already deleted l2

num1 = [2, 5, 8, 1, 4]

o/p

[4, 25, 64, 1, 16]

```
num1 = [2,5,8,1,4]
ns = []
for i in num1:
    s = i*i
    ns.append(s)
ns
```

[4,25,64,1,16] - o/p

- len (ns) # to find the number of total element

5 - o/p

ns. count (64) # to find out how many times 64 is present

o/p - 1

Program:

1) Create, even, odd, prime number list from 1 to 20 no

```
en = []
on = []
pn = []
for i in range(1,21,1):
    if(i%2==0):
```

```
    en.append(i)
else:
    on.append(i)
for j in range(2,i,1):
    if (i%j==0):
        break
else:
    pn.append(i)

Print(en,on,pn)
```

we used only 1 loop
i.e nested for loop.

o/p: [2,4,6,8,10,12,14,16,18,20] [1,3,5,7,9,11,13,15,17,19] [1,2,3,
5,7,11,13,17,19]

```
else:
    if (i>1):
        pn.append(i)
```

then u get
[2,3,5,7,11,13,17,19] for
Prime no.